

Un servidor web en Python puro, una base de datos PostgreSQL que crea sus propios usuarios y un Ubuntu que se actualiza y da de alta usuarios desde un archivo en la misma carpeta. Cada ejemplo vive en su propio directorio y se construye con `podman build`.

```
$ sudo dnf install -y podman && podman --version
```

[Webserver Python](#)[Base de datos](#)[Ubuntu con usuarios](#)[Notas RHEL](#)

Webserver con Python puro

Solo librería estándar (`http.server`), sin Flask ni dependencias. Imagen base UBI 9 de Red Hat, que no requiere suscripción y corre como usuario no root.

```
web-python/  
├─ Containerfile  
└─ server.py
```

server.py[Copiar](#)

```
from http.server import HTTPServer, BaseHTTPRequestHandler  
import datetime  
import json  
import os  
import socket  
  
class Handler(BaseHTTPRequestHandler):  
    def do_GET(self):  
        if self.path == "/health":  
            body = json.dumps({"status": "ok"}).encode()  
            ctype = "application/json"  
        else:  
            html = (  
                "<h1>Hola desde Podman</h1>"  
                f"<p>Contenedor: {socket.gethostname()}</p>"  
                f"<p>Hora: {datetime.datetime.now():%Y-%m-%d %H:%M:%S}</p>"  
            )  
            body = html.encode()  
            ctype = "text/html; charset=utf-8"  
  
        self.send_response(200)
```

```
self.send_header("Content-Type", ctype)
self.send_header("Content-Length", str(len(body)))
self.end_headers()
self.wfile.write(body)

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 8080))
    print(f"Escuchando en 0.0.0.0:{port}")
    HTTPServer(("0.0.0.0", port), Handler).serve_forever()
```

Containerfile

[Copiar](#)

```
FROM registry.access.redhat.com/ubi9/python-311

LABEL description="Webserver en Python puro con http.server"

WORKDIR /opt/app-root/src

# Copiamos el código con el usuario no root de la imagen (1001)
COPY --chown=1001:0 server.py .

ENV PORT=8080 \
    PYTHONUNBUFFERED=1

EXPOSE 8080
USER 1001

CMD ["python", "server.py"]
```

Construir y probar

terminal

[Copiar](#)

```
cd web-python
podman build -t py-web:1.0 .
podman run -d --name web -p 8080:8080 py-web:1.0

curl http://localhost:8080
curl http://localhost:8080/health
podman logs web
```

Base de datos PostgreSQL con usuarios

El script SQL se copia a `/docker-entrypoint-initdb.d/` y se ejecuta la primera vez que arranca el contenedor: crea dos usuarios con permisos distintos, una base, una tabla y datos de prueba.

db-postgres/
├─ Containerfile
└─ init.sql

init.sql

[Copiar](#)

```
-- Usuarios
CREATE USER app_user WITH PASSWORD 'App2026!';
CREATE USER reporte  WITH PASSWORD 'Rep2026!';

-- Base de datos cuyo dueño es app_user
CREATE DATABASE inventario OWNER app_user;

\connect inventario

CREATE TABLE productos (
    id          SERIAL PRIMARY KEY,
    nombre      VARCHAR(100) NOT NULL,
    cantidad    INTEGER      NOT NULL DEFAULT 0,
    creado      TIMESTAMP     NOT NULL DEFAULT now()
);
ALTER TABLE productos OWNER TO app_user;

INSERT INTO productos (nombre, cantidad) VALUES
    ('Servidor ProLiant', 4),
    ('Switch 48p', 2),
    ('Disco NVMe 3.84TB', 12);

-- reporte: solo lectura
GRANT CONNECT ON DATABASE inventario TO reporte;
GRANT USAGE    ON SCHEMA public      TO reporte;
GRANT SELECT   ON ALL TABLES IN SCHEMA public TO reporte;
ALTER DEFAULT PRIVILEGES FOR ROLE app_user IN SCHEMA public
    GRANT SELECT ON TABLES TO reporte;
```

Containerfile

[Copiar](#)

```
FROM docker.io/library/postgres:16

LABEL description="PostgreSQL con usuarios y datos iniciales"

# Se ejecuta solo si el directorio de datos está vacío
```

```
# Se ejecuta solo si el directorio de datos está vacío
COPY init.sql /docker-entrypoint-initdb.d/01-init.sql

ENV TZ=America/Mexico_City

EXPOSE 5432
```

Construir y probar

terminal

Copiar

```
cd db-postgres
podman build -t pg-usuarios:1.0 .
podman volume create pgdata

# La contraseña del superusuario se pasa en ejecución, no en la imagen
podman run -d --name db \
  -e POSTGRES_PASSWORD='Admin2026!' \
  -p 5432:5432 \
  -v pgdata:/var/lib/postgresql/data \
  pg-usuarios:1.0

podman logs -f db          # espera "ready to accept connections"

# Ver usuarios creados
podman exec -it db psql -U postgres -c '\du'

# Consultar como app_user y como reporte
podman exec -it db psql -U app_user -d inventario -c 'SELECT * FROM productos'
podman exec -it db psql -U reporte -d inventario -c 'SELECT count(*) FROM pr

# reporte no puede escribir (debe fallar)
podman exec -it db psql -U reporte -d inventario \
  -c "INSERT INTO productos(nombre) VALUES ('prueba');"

```

Ubuntu que se actualiza y crea usuarios desde un archivo

El archivo `usuarios.txt` está en la misma ruta que el Containerfile (el contexto de build). Durante la construcción se actualiza el sistema, se lee el archivo línea por línea y se crea cada usuario en el grupo `devops` con `sudo`.

ubuntu-usuarios/

└─ Containerfile
└─ usuarios.txt

usuarios.txt formato usuario:contraseña

Copiar

```
# Las líneas con # y las vacías se ignoran
daniel:Daniel2026!
ana:Ana2026!
alumno1:Alumno2026!
alumno2:Alumno2026!
```

Containerfile

Copiar

```
FROM docker.io/library/ubuntu:24.04

LABEL description="Ubuntu actualizado con usuarios desde usuarios.txt"

ENV DEBIAN_FRONTEND=noninteractive \
    TZ=America/Mexico_City

# 1. Actualizar el sistema operativo e instalar utilerías básicas
RUN apt-get update && \
    apt-get -y upgrade && \
    apt-get install -y --no-install-recommends sudo tzdata vim-tiny iputils-ping \
    apt-get clean && rm -rf /var/lib/apt/lists/*

# 2. Copiar el archivo que está en la misma ruta que este Containerfile
COPY usuarios.txt /tmp/usuarios.txt

# 3. Crear grupo y usuarios leyendo el archivo
RUN groupadd -f devops && \
    while IFS=: read -r usuario clave || [ -n "$usuario" ]; do \
        case "$usuario" in ''|' #'*) continue ;; esac; \
        useradd -m -s /bin/bash -G devops "$usuario"; \
        echo "$usuario:$clave" | chpasswd; \
        echo "Usuario creado: $usuario"; \
    done < /tmp/usuarios.txt && \
    echo '%devops ALL=(ALL) NOPASSWD:ALL' > /etc/sudoers.d/devops && \
    chmod 440 /etc/sudoers.d/devops && \
    rm -f /tmp/usuarios.txt

WORKDIR /home

CMD ["/bin/bash"]
```

Construir y probar

terminal

Copiar

```
cd ubuntu-usuarios
podman build -t ubuntu-usuarios:1.0 .

# Verificar usuarios y grupo sin entrar al contenedor
podman run --rm ubuntu-usuarios:1.0 getent group devops
podman run --rm ubuntu-usuarios:1.0 ls /home

# Sesión interactiva: cambiar a un usuario y probar sudo
podman run -it --rm --name ubu ubuntu-usuarios:1.0
  su - daniel
  sudo apt list --upgradable
```

Notas para Podman en RHEL

- Podman busca Containerfile o Dockerfile en el directorio; ambos nombres funcionan igual. Para otro nombre usa `-f archivo`.
- Usa nombres de imagen completos (`docker.io/library/...` , `registry.access.redhat.com/...`) para evitar el prompt de short-names de `/etc/containers/registries.conf`.
- En modo rootless no puedes publicar puertos menores a 1024; por eso los ejemplos usan 8080 y 5432.
- Si montas carpetas del host (bind mounts) con SELinux en enforcing, agrega `:Z` al volumen, por ejemplo `-v ./datos:/data:Z`.
- Para acceder desde otra máquina abre el puerto: `sudo firewall-cmd --add-port=8080/tcp --permanent && sudo firewall-cmd --reload`.
- Las contraseñas en `usuarios.txt` e `init.sql` quedan dentro de la imagen: sirven para laboratorio, no para producción. En producción usa `podman secret` o variables en ejecución.
- Limpieza: `podman rm -f web db && podman volume rm pgdata && podman image prune`.

Probado con la sintaxis de Podman 4.x/5.x en RHEL 9 y derivados (Rocky, Alma).

